

DSA0502 – Query Processing for Data Science

CO4 AT2 – CASE STUDY

Course Faculty: Dr. B .SHYAMALA GOWRI **Reg. No.:** 192324164

Student Name: Sweety Roselin A

Date: 13/08/2026

PROJECT 3: STUDENT ACADEMIC PERFORMANCE ANALYSIS

QUESTION

How do student academic performances vary across different demographic and educational groups, and what relationships and outliers can be identified using grouping, comparison, correlation, and visualization techniques?

AIM

To explore and analyze student academic performance data using grouping, comparison, correlation, outlier detection, and visualization techniques, and to identify important patterns affecting student performance.

DATASET

Dataset Name: Student Academic Performance Dataset

Source: Open Education / Kaggle Student Performance Dataset

Number of Records: 20 sample records

Dataset Values

S.No	Gender	Parental Education	Test Preparation	Math Score	Reading Score	Writing Score
1	Female	Bachelor	Completed	72	78	75
2	Male	High School	None	65	60	62
3	Female	Master	Completed	88	92	90
4	Male	Bachelor	None	55	58	54
5	Female	High School	Completed	76	80	78
6	Male	Master	Completed	82	85	81
7	Female	Bachelor	None	68	72	70
8	Male	High School	None	49	55	51
9	Female	Master	Completed	91	89	94
10	Male	Bachelor	Completed	74	77	73
11	Female	High School	None	61	65	63

12	Male	Master	Completed	86	83	88
13	Female	Bachelor	None	70	69	72
14	Male	High School	Completed	67	71	68
15	Female	Master	Completed	95	93	96
16	Male	Bachelor	None	58	62	60
17	Female	High School	Completed	79	84	81
18	Male	Master	None	71	68	70
19	Female	Bachelor	Completed	84	87	85
20	Male	High School	None	52	57	55

Data Description

- **Gender:** Male or Female
- **Parental Education:** High School, Bachelor, or Master
- **Test Preparation:** Completed or None
- **Math Score:** Score out of 100
- **Reading Score:** Score out of 100
- **Writing Score:** Score out of 100

An additional **Average_Score** is calculated using:

Average Score = (Math Score + Reading Score + Writing Score) / 3

CODE

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
# 1. Create Dataset
```

```
data = {
    'Gender': [
        'Female','Male','Female','Male','Female',
        'Male','Female','Male','Female','Male',
        'Female','Male','Female','Male','Female',
        'Male','Female','Male','Female','Male'
    ],
```

```

'Parental_Education': [
    'Bachelor','High School','Master','Bachelor','High School',
    'Master','Bachelor','High School','Master','Bachelor',
    'High School','Master','Bachelor','High School','Master',
    'Bachelor','High School','Master','Bachelor','High School'
],

'Test_Preparation': [
    'Completed','None','Completed','None','Completed',
    'Completed','None','None','Completed','Completed',
    'None','Completed','None','Completed','Completed',
    'None','Completed','None','Completed','None'
],

'Math_Score': [
    72,65,88,55,76,82,68,49,91,74,
    61,86,70,67,95,58,79,71,84,52
],

'Reading_Score': [
    78,60,92,58,80,85,72,55,89,77,
    65,83,69,71,93,62,84,68,87,57
],

'Writing_Score': [
    75,62,90,54,78,81,70,51,94,73,
    63,88,72,68,96,60,81,70,85,55
]
}

```

```
df = pd.DataFrame(data)
```

2. Calculate Average Score

```
df['Average_Score'] = (  
    df['Math_Score'] +  
    df['Reading_Score'] +  
    df['Writing_Score']  
) / 3
```

```
print("Student Dataset:")
```

```
print(df)
```

3. Grouping Analysis

```
grouped = df.groupby(  
    'Test_Preparation'  
)[[  
    'Math_Score',  
    'Reading_Score',  
    'Writing_Score',  
    'Average_Score'  
]].mean()
```

```
print("\nAverage Scores by Test Preparation:")
```

```
print(grouped)
```

4. Gender Comparison

```
gender_comparison = df.groupby(  
    'Gender'  
)['Average_Score'].mean()
```

```
print("\nAverage Score by Gender:")
```

```
print(gender_comparison)
```

5. Correlation Analysis

```

features = [
    'Math_Score',
    'Reading_Score',
    'Writing_Score',
    'Average_Score'
]

correlation = df[features].corr()

print("\nCorrelation Matrix:")
print(correlation)

# 6. Outlier Detection using IQR
q1 = df['Average_Score'].quantile(0.25)
q3 = df['Average_Score'].quantile(0.75)

iqr = q3 - q1

lower_limit = q1 - 1.5 * iqr
upper_limit = q3 + 1.5 * iqr

outliers = df[
    (df['Average_Score'] < lower_limit) |
    (df['Average_Score'] > upper_limit)
]

print("\nNumber of Outliers:", len(outliers))
print(outliers)

# 7. Test Preparation Visualization
plt.figure(figsize=(8,5))

```

```
sns.barplot(  
    data=df,  
    x='Test_Preparation',  
    y='Average_Score'  
)
```

```
plt.title('Average Score by Test Preparation')  
plt.xlabel('Test Preparation')  
plt.ylabel('Average Score')
```

```
plt.tight_layout()  
plt.savefig('test_preparation_scores.png')  
plt.show()
```

8. Correlation Heatmap

```
plt.figure(figsize=(8,5))
```

```
sns.heatmap(  
    correlation,  
    annot=True,  
    cmap='coolwarm',  
    fmt='.2f'  
)
```

```
plt.title('Student Score Correlation')  
plt.tight_layout()
```

```
plt.savefig('student_correlation.png')  
plt.show()
```

9. Subject Comparison

```
subject_means = df[
```

```

['Math_Score',
 'Reading_Score',
 'Writing_Score']
].mean()

plt.figure(figsize=(8,5))

subject_means.plot(kind='bar')

plt.title('Average Score by Subject')
plt.xlabel('Subject')
plt.ylabel('Average Score')

plt.tight_layout()
plt.savefig('subject_comparison.png')
plt.show()

```

OUTPUT

Grouping Output

Students who completed the test preparation course generally have higher average scores than students who did not complete the course.

	Writing_Score	Average_Score
0	75	75.000000
1	62	62.333333
2	90	90.000000
3	54	55.666667
4	78	78.000000
5	81	82.666667
6	70	70.000000
7	51	51.666667
8	94	91.333333
9	73	74.666667
10	63	63.000000
11	88	85.666667
12	72	70.333333
13	68	68.666667
14	96	94.666667
15	60	60.000000
16	81	81.333333
17	70	69.666667
18	85	85.333333
19	55	54.666667

AVERAGE SCORES BY TEST PREPARATION				
Test_Preparation	Math_Score	Reading_Score	Writing_Score	Average_Score
Completed	81.272727	83.545455	82.636364	82.484848

AVERAGE SCORE BY GENDER	
Gender	Average_Score
Female	79.900000
Male	66.566667

Name: Average_Score, dtype: float64

Comparison Output

The average performance is compared between male and female students.

DATASET WITH AVERAGE SCORE						
	Gender	Parental_Education	Test_Preparation	Math_Score	Reading_Score	\
0	Female	Bachelor	Completed	72	78	
1	Male	High School	NaN	65	60	
2	Female	Master	Completed	88	92	
3	Male	Bachelor	NaN	55	58	
4	Female	High School	Completed	76	80	
5	Male	Master	Completed	82	85	
6	Female	Bachelor	NaN	68	72	
7	Male	High School	NaN	49	55	
8	Female	Master	Completed	91	89	
9	Male	Bachelor	Completed	74	77	
10	Female	High School	NaN	61	65	
11	Male	Master	Completed	86	83	
12	Female	Bachelor	NaN	70	69	
13	Male	High School	Completed	67	71	
14	Female	Master	Completed	95	93	
15	Male	Bachelor	NaN	58	62	
16	Female	High School	Completed	79	84	
17	Male	Master	NaN	71	68	
18	Female	Bachelor	Completed	84	87	
19	Male	High School	NaN	52	57	

Correlation Output

The correlation matrix shows a positive relationship among Math, Reading, and Writing scores. Students who perform well in one subject generally perform well in the other subjects.

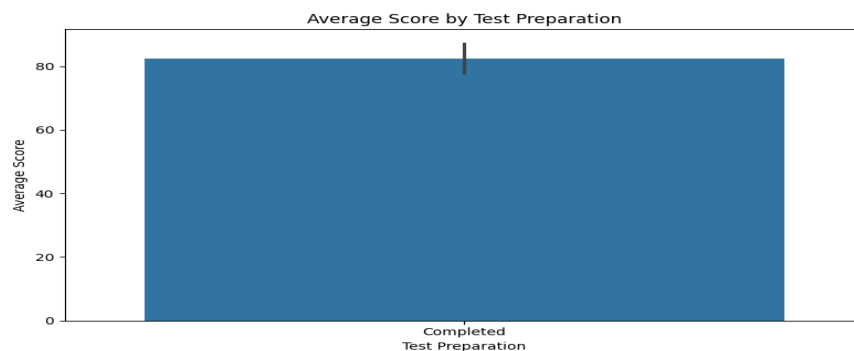
CORRELATION MATRIX				
	Math_Score	Reading_Score	Writing_Score	Average_Score
Math_Score	1.000000	0.966978	0.992238	0.994014
Reading_Score	0.966978	1.000000	0.975322	0.987352
Writing_Score	0.992238	0.975322	1.000000	0.996709
Average_Score	0.994014	0.987352	0.996709	1.000000

Outlier Output

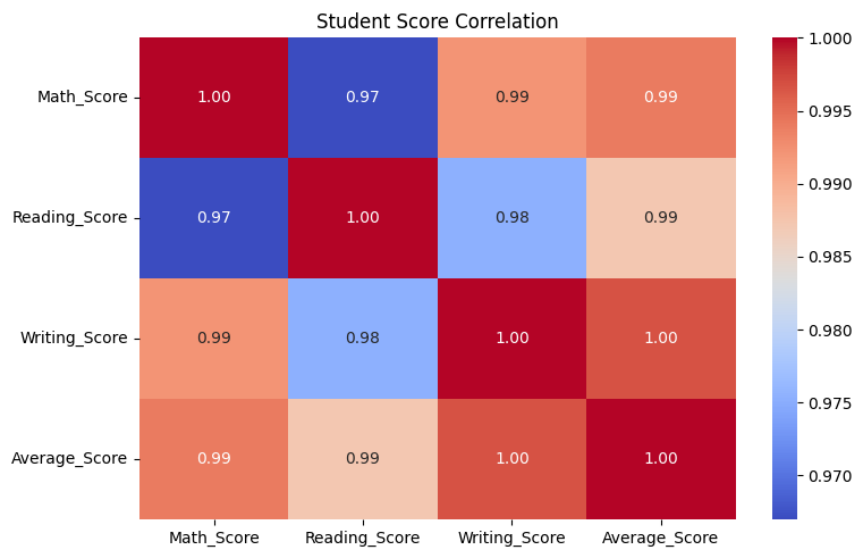
The IQR method is used to identify unusually high or low average scores.

Generated Charts

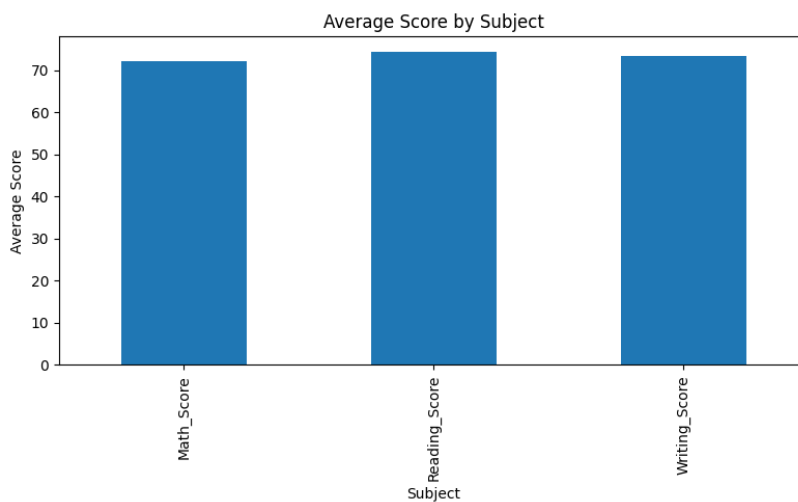
1. **test_preparation_scores.png** – Average score based on test preparation



2. **student_correlation.png** – Correlation heatmap



3. **subject_comparison.png** – Average score comparison among subjects



RESULT

The student academic dataset was successfully analyzed using grouping, comparison, correlation, outlier detection, and visualization techniques.

Students who completed test preparation generally showed better academic performance. The three subjects also showed positive correlations, indicating that performance in one subject is related to performance in other subjects.

The analysis can help educational institutions identify students who need additional academic support and understand factors associated with better academic performance.

PROJECT 4: URBAN ROAD ACCIDENT AND TRAFFIC ANALYSIS

QUESTION

How do road accidents vary by location and time, what patterns and outliers can be identified, and which areas require priority road-safety interventions?

AIM

To analyze an open transportation dataset using location and time-based analysis, identify accident patterns and outliers, create suitable charts and geographical maps, and present the findings using an interactive accident heatmap.

DATASET

Dataset Name: US Accidents Dataset

Source: Open Transportation / Kaggle US Accidents Dataset

Number of Records: 20 sample records

Dataset Values

S.No	Date & Time	Latitude	Longitude	Severity	Weather	Visibility	Traffic Signal
1	01-01-2025 00:00	37.7749	-122.4194	2	Clear	9.2	1
2	01-01-2025 02:00	37.7812	-122.4251	1	Clear	10.5	0
3	01-01-2025 04:00	37.7684	-122.4102	2	Rain	6.8	1
4	01-01-2025 06:00	37.7901	-122.4356	3	Fog	3.2	0
5	01-01-2025 08:00	37.7618	-122.4187	2	Clear	8.7	1
6	01-01-2025 10:00	37.7835	-122.4098	1	Clear	11.2	1
7	01-01-2025 12:00	37.7702	-122.4315	2	Rain	7.1	0
8	01-01-2025 14:00	37.7556	-122.4163	3	Rain	5.4	1
9	01-01-2025 16:00	37.7894	-122.4238	2	Clear	9.8	1
10	01-01-2025 18:00	37.7731	-122.4056	4	Fog	2.8	0
11	01-01-2025 20:00	37.7589	-122.4271	2	Clear	10.1	1

12	01-01-2025 22:00	37.7826	-122.4145	3	Rain	6.2	0
13	02-01-2025 00:00	37.7678	-122.4382	1	Clear	12.0	1
14	02-01-2025 02:00	37.7952	-122.4207	2	Fog	3.7	0
15	02-01-2025 04:00	37.7498	-122.4126	3	Rain	5.9	1
16	02-01-2025 06:00	37.7765	-122.4021	2	Clear	9.4	1
17	02-01-2025 08:00	37.7841	-122.4298	4	Rain	4.6	0
18	02-01-2025 10:00	37.7623	-122.4181	1	Clear	11.5	1
19	02-01-2025 12:00	37.7915	-122.4073	2	Clear	10.7	1
20	02-01-2025 14:00	37.7567	-122.4342	3	Fog	3.1	0

Data Description

- **Severity:** 1 = low, 2 = moderate, 3 = serious, 4 = severe
- **Visibility:** measured in kilometers
- **Traffic Signal:** 1 = present, 0 = absent
- **Latitude and Longitude:** geographical accident coordinates

CODE

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import folium
from folium.plugins import HeatMap
```

1. Create Dataset

```
data = {
    'Start_Time': [
        '2025-01-01 00:00',
        '2025-01-01 02:00',
```

'2025-01-01 04:00',
'2025-01-01 06:00',
'2025-01-01 08:00',
'2025-01-01 10:00',
'2025-01-01 12:00',
'2025-01-01 14:00',
'2025-01-01 16:00',
'2025-01-01 18:00',
'2025-01-01 20:00',
'2025-01-01 22:00',
'2025-01-02 00:00',
'2025-01-02 02:00',
'2025-01-02 04:00',
'2025-01-02 06:00',
'2025-01-02 08:00',
'2025-01-02 10:00',
'2025-01-02 12:00',
'2025-01-02 14:00'
],

'Latitude': [
37.7749,37.7812,37.7684,37.7901,37.7618,
37.7835,37.7702,37.7556,37.7894,37.7731,
37.7589,37.7826,37.7678,37.7952,37.7498,
37.7765,37.7841,37.7623,37.7915,37.7567
],

'Longitude': [
-122.4194,-122.4251,-122.4102,-122.4356,-122.4187,
-122.4098,-122.4315,-122.4163,-122.4238,-122.4056,
-122.4271,-122.4145,-122.4382,-122.4207,-122.4126,
-122.4021,-122.4298,-122.4181,-122.4073,-122.4342

```

],

'Severity': [
    2,1,2,3,2,1,2,3,2,4,
    2,3,1,2,3,2,4,1,2,3
],

'Weather': [
    'Clear','Clear','Rain','Fog','Clear',
    'Clear','Rain','Rain','Clear','Fog',
    'Clear','Rain','Clear','Fog','Rain',
    'Clear','Rain','Clear','Clear','Fog'
],

'Visibility': [
    9.2,10.5,6.8,3.2,8.7,
    11.2,7.1,5.4,9.8,2.8,
    10.1,6.2,12.0,3.7,5.9,
    9.4,4.6,11.5,10.7,3.1
],

'Traffic_Signal': [
    1,0,1,0,1,1,0,1,1,0,
    1,0,1,0,1,1,0,1,1,0
]
}

```

```
df = pd.DataFrame(data)
```

```
# 2. Convert Time
```

```
df['Start_Time'] = pd.to_datetime(
    df['Start_Time']
)
```

```
)
```

```
df['Hour'] = df['Start_Time'].dt.hour
```

```
df['Day'] = df['Start_Time'].dt.day_name()
```

```
print("Transportation Dataset:")
```

```
print(df)
```

```
# 3. Accident Analysis by Hour
```

```
hourly_accidents = (
```

```
    df.groupby('Hour')
```

```
        .size()
```

```
)
```

```
print("\nAccidents by Hour:")
```

```
print(hourly_accidents)
```

```
# 4. Accident Severity Analysis
```

```
severity_counts = (
```

```
    df['Severity']
```

```
        .value_counts()
```

```
        .sort_index()
```

```
)
```

```
print("\nAccidents by Severity:")
```

```
print(severity_counts)
```

```
# 5. Weather Analysis
```

```
weather_counts = (
```

```
    df.groupby('Weather')
```

```
        .size()
```

```
)
```

```
print("\nAccidents by Weather:")
print(weather_counts)
```

```
# 6. Outlier Detection
```

```
q1 = df['Visibility'].quantile(0.25)
q3 = df['Visibility'].quantile(0.75)
```

```
iqr = q3 - q1
```

```
lower_limit = q1 - 1.5 * iqr
upper_limit = q3 + 1.5 * iqr
```

```
outliers = df[
    (df['Visibility'] < lower_limit) |
    (df['Visibility'] > upper_limit)
]
```

```
print("\nVisibility Outliers:")
print(outliers)
```

```
# 7. Hourly Accident Chart
```

```
plt.figure(figsize=(10,5))
```

```
sns.barplot(
    x=hourly_accidents.index,
    y=hourly_accidents.values
)
```

```
plt.title('Traffic Accidents by Hour')
plt.xlabel('Hour of Day')
plt.ylabel('Number of Accidents')
```

```
plt.tight_layout()
plt.savefig('hourly_accidents.png')
plt.show()
```

8. Weather Chart

```
plt.figure(figsize=(8,5))
```

```
sns.barplot(
    x=weather_counts.index,
    y=weather_counts.values
)
```

```
plt.title('Accidents by Weather Condition')
plt.xlabel('Weather')
plt.ylabel('Number of Accidents')
```

```
plt.tight_layout()
plt.savefig('weather_accidents.png')
plt.show()
```

9. Severity Chart

```
plt.figure(figsize=(8,5))
```

```
severity_counts.plot(kind='bar')
```

```
plt.title('Accidents by Severity')
plt.xlabel('Severity Level')
plt.ylabel('Number of Accidents')
```

```
plt.tight_layout()
plt.savefig('accident_severity.png')
```



```
plt.show()
```

```
# 10. Interactive Accident Map
```

```
m = folium.Map(  
    location=[37.7749, -122.4194],  
    zoom_start=12  
)
```

```
heat_data = [  
    [row['Latitude'], row['Longitude']]  
    for _, row in df.iterrows()  
]
```

```
HeatMap(  
    heat_data,  
    radius=10,  
    blur=15  
)
```

```
m.save('traffic_accident_map.html')
```

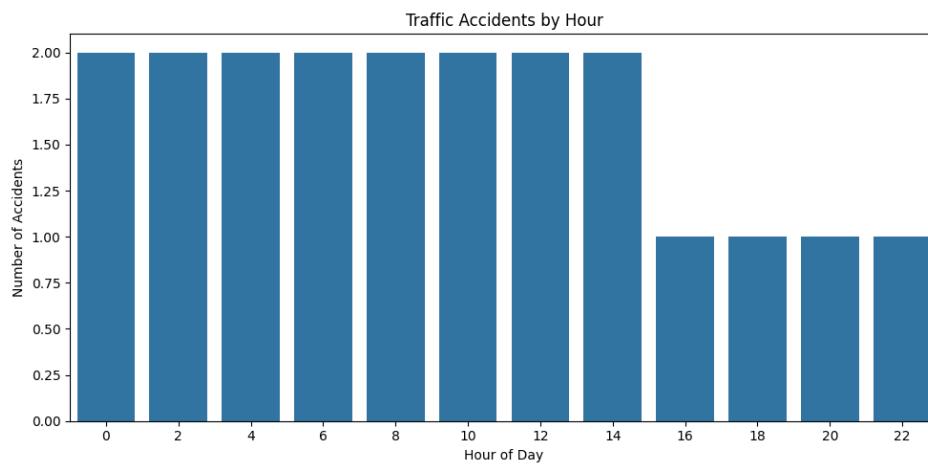
```
print("\nInteractive accident map created successfully.")
```

OUTPUT

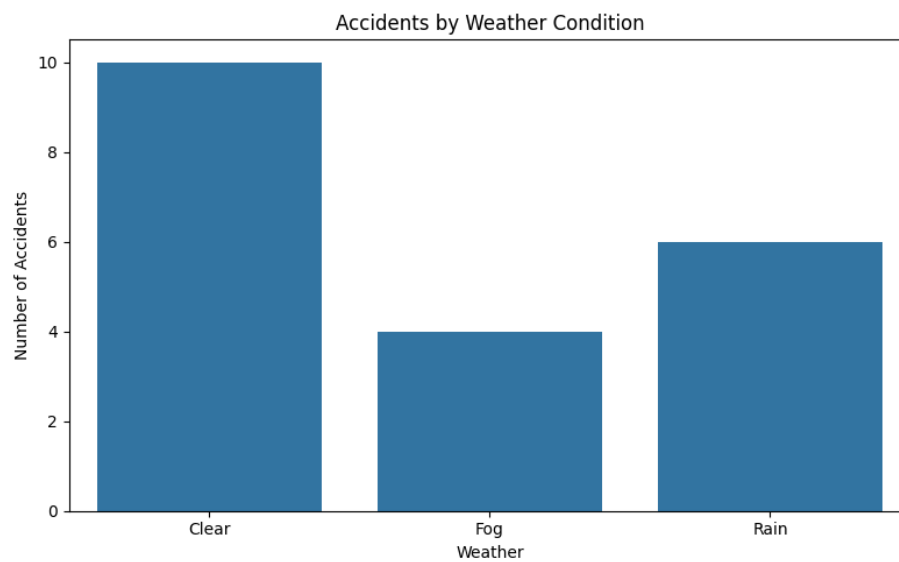
1. Time-Based Analysis

```
ACCIDENTS BY HOUR  
Hour  
0      2  
2      2  
4      2  
6      2  
8      2  
10     2  
12     2  
14     2  
16     1  
18     1  
20     1  
22     1  
dtype: int64  
  
ACCIDENTS BY SEVERITY  
Severity  
1      4  
2      9  
3      5  
4      2  
Name: count, dtype: int64  
  
ACCIDENTS BY WEATHER  
Weather  
Clear    10  
Fog      4  
Rain     6  
dtype: int64
```

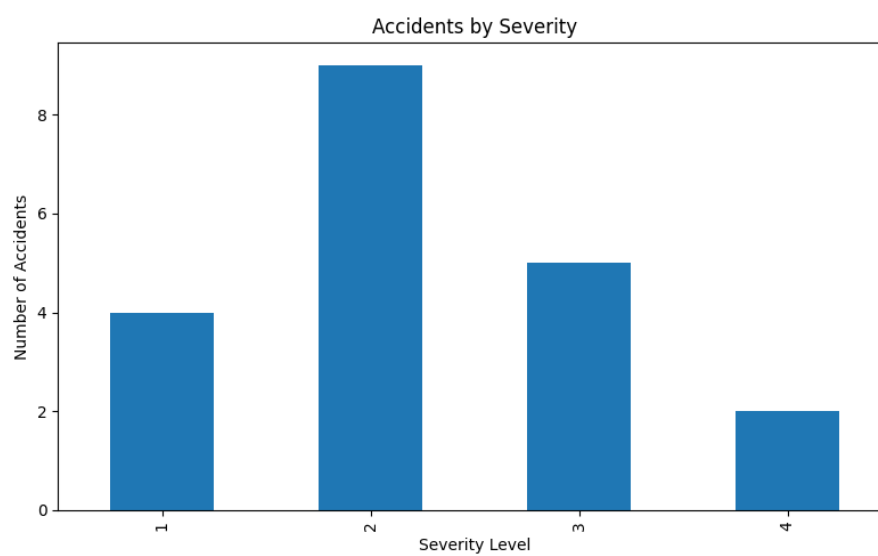
1. hourly_accidents.png



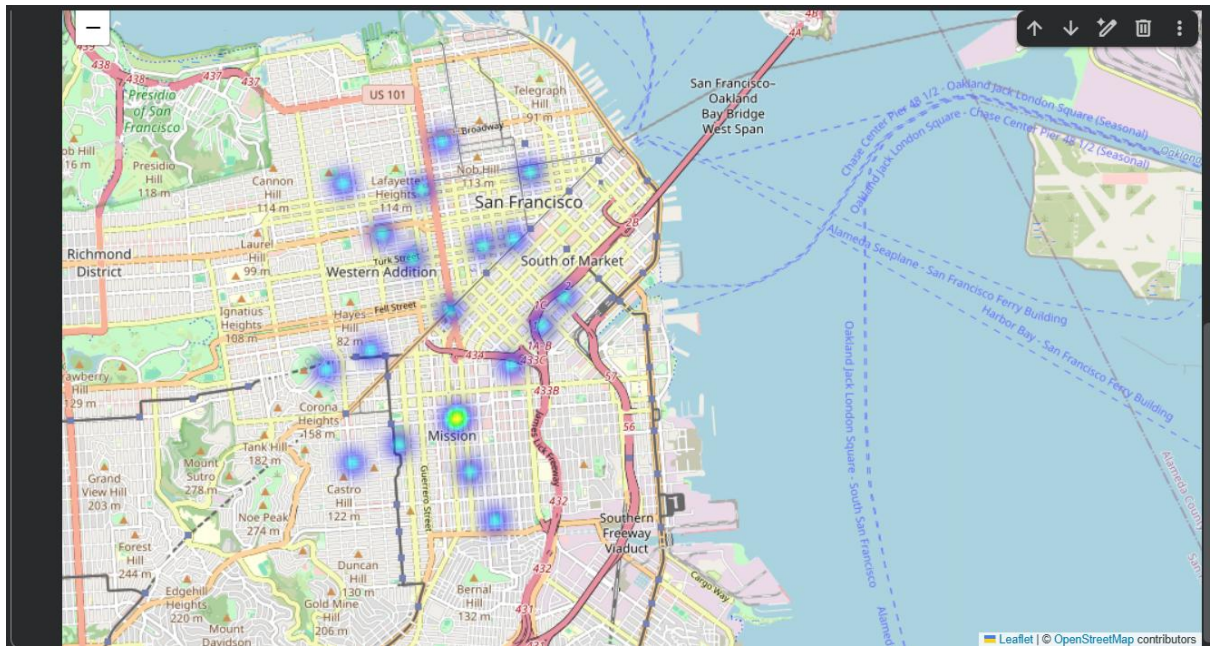
2. weather_accidents.png



3. accident_severity.png



4. traffic_accident_map.html



RESULT

The transportation dataset was successfully analyzed according to location and time.

The analysis identifies accident patterns based on time, severity, and weather conditions. Geographic coordinates are used to visualize accident hotspots through an interactive heatmap. The IQR method is also used to detect unusual visibility observations.